

Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barengi

Last (family) Name _____
First (given) Name _____
Matricola _____
Codice Persona _____

Professor: ☐ Zanero ☐ Carminati ☐ Barengi
Have you done any challenges/homework? ☐ Yes ☐ No
Do you want to withdraw? ☐ Yes ☐ No
Priority Grading ☐ Yes ☐ No
Motivation:

Instructions

- **Duration:** 2.5 hours.
- The exam is composed of 15 pages. Check that you have all of them. No extra paper is allowed.
- The exam is "closed book." You are not allowed to consult any course material and notes.
- No extra devices (e.g., phones, iPad) are allowed. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found, and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- **All responses must be written in pen (any color except red). Answers written in pencil may not be legible when scanned and, therefore, may not be graded.**
- Always motivate your answer.
- If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).

DISTANCE MODE ONLY

- On the PC, you can open only the forms browser tab and the exam pdf. Any other application/document is prohibited.
- You will have to share your screen, and leave your microphone opened, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct exam delivery.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room, and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or a subset at the instructor's decision.
- Regarding the submission:
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
 - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
 - We will evaluate only the uploaded **readable** documents.
- **USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT** (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)



Amazon, a logistics company, uses a fleet of autonomous robots (designed and produced in-house) to transport pallets within a warehouse. The robots navigate using QR codes on the floor and receive orders from a "Central Command" server via local Wi-Fi. The robots act on any command signed with a valid HMAC. All robots share the same secret key for HMAC verification, simplifying deployment. To update the robot firmware, an administrator must plug a USB drive into the robot. The robot automatically executes any file named update.exe

found on the USB drive when it is inserted. The Central Command server is air-gapped from the Internet but connected to the corporate IT network via a firewall that allows traffic on port 69 for "inventory reporting."

1.1 [1.5 points] Identify and describe the 2 most relevant assets at risk in such a scenario, along with their associated threats and threat agents. Please insert just one element per column (Asset, Threat, Threat Agent); additional elements may be considered as errors.

Asset	Threat	Threat Agent
Operational Continuity (Warehouse Throughput)	Threat: Denial of Service (DoS) or sabotage where robots are commanded to collide or stop working, halting shipments.	Competitors / Disgruntled Employees
Motivation Financial damage. Sabotage to gain market share or revenge.		
Proprietary Logistics Data	Threat: Exfiltration of inventory levels and shipment destinations via the port 80 link.	Corporate Spies
Motivation Gaining insight into the company's supply chain strategy. Industrial espionage.		

2. [1.5 points] Identify 2 potential vulnerabilities in the company's operations and specify a countermeasure.

Vulnerability	Exploit	Countermeasure

Shared HMAC Key across all robots	If an attacker physically steals or compromises <i>one</i> robot, they extract the key and can forge commands for <i>all</i> robots in the fleet.	Unique Keys per Device: Provision each robot with a unique cryptographic key so that compromising one does not compromise the whole fleet.
Automatic execution from USB (update.exe)	An attacker (e.g., a cleaner or insider) inserts a malicious USB drive. The robot runs the malware immediately without verification, leading to full system compromise.	Code Signing & Disable Auto-Run: Require firmware updates to be digitally signed by the vendor and disable automatic execution of files from external media.

3. [1 point] The company considers replacing the HMAC key system with Biometric Authentication (Face Recognition) for the human operators who override the robots. Discuss the advantages and disadvantages in this specific context.

Advantages
Non-repudiation: It is harder for an operator to deny they issued a command if their face was scanned (unlike a shared password). Speed: Quick access for supervisors in emergency situations without typing complex passwords.
Disadvantages
Environmental factors: Warehouses may be poorly lit, or operators may wear safety gear (goggles, masks) that obstruct facial recognition. Privacy: Storing biometric data of employees requires strict compliance with privacy laws, adding legal/compliance overhead.

Exercise 2 - Introduction to Cryptography (6 points)

[2 points] Assume that your company needs to manage a large document archive, where past documents are indexed and should be available, unmodified, with a proof of authorship to anyone retrieving them. Describe synthetically which cryptographic tools need to be used, and state at least one technical challenge in maintaining this infrastructure.

The question is asking to preserve the availability, integrity, and origin authentication properties of the documents. The origin authentication property of the documents can only be guaranteed with a digital signature. Digitally signing all the documents before archiving also guarantees their integrity, as a digital signature is only valid for the message it is computed on.

One, if not the main, technical challenge of a digital signature based approach is to manage the key pairs for signing and verification on a large document archive: a viable option is the one of building a PKI dedicated for the archive itself.

[4 points] Consider the following password generation schemes, and quantify their security margin against both an attacker which assumes that the passwords are uniformly randomly sampled from the set of admissible ones, and one which instead guesses knowing the all the information provided below

1. A password is composed by 4 words uniformly drawn from a 32768 words dictionary containing only 6-characters lowercase words in English (26 characters alphabet)
2. A password is composed by 4 words uniformly drawn from a 32768 words dictionary containing only 6-characters lowercase words in Russian (consider the 33 characters Cyrillic alphabet)
3. A password is composed by the byte sequence obtained concatenating 4 images from a catalog containing 2 MiB ($= 2^{20}$ B) images. The catalog contains 10004 images, 10000 are non-cat animals, and 4 are images of cats. The password is composed of 3 cat images and a uniformly random animal image.

1. A diceware approach on a 2^{15} words dictionary yields 15b of information per word, a 4 words password requires 2^{60} attempts to be bruteforced (2^{59} on average). Knowing the fact that the distribution is uniform yields no advantage (it's the worst case already).
2. Same as point 1. The encoding of the elements of the dictionary is not relevant
3. Assuming a uniform distribution over the images, it's a diceware approach with a 10004-sized dictionary. Each draw contains $\log_2(10004) \sim 13.28$ b of information; guessing a 4 image password will require $\sim 2^{53.15}$ attempts. Knowing the distribution reduces the effective password space to $4^3 * \text{binom}(4,3) * 10004 = 64 * 4 * 10004 = 2561024 \sim 2^{21}$.

Exercise 3 - Binary Vulnerabilities (6 points)

```
1. #include ...
2.
3. typedef struct {
4.     unsigned int size;
5.     char alert[40];
6. } Log;
7.
8. int main(void) {
9.     Log log = {0};
10.
11.     puts("=== Alert Logger ===");
12.
13.     printf("Alert size: ");
14.     scanf("%u", &log.size);
15.
16.     printf("Alert message: ");
17.     read(0, log.alert, log.size); // Reads log.size bytes from stdin into log.alert
18.
19.     if (log.size > sizeof(log.alert)) { // sizeof is computed at compile time
20.         puts("Alert too long, discarded.");
21.         exit(1);
22.     }
23.
24.     puts("Alert stored:");
25.     printf(log.alert);
26.
27.     return 0;
28. }
```

The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdec1** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors are present (unless specified otherwise).

[2 points] 1. Fill out the following table by reporting all the vulnerabilities you can spot in the code. If you believe there might be a vulnerability, specify for each vulnerability the **corresponding line/s of code** and specify if it is possible to exploit it.

Vulnerability (e.g., format string, buffer overflow, ...)	Line	<u>Is it exploitable?</u>	
		YES	NO
[1 point] Buffer Overflow (No due to exit)	17	☐	☒
[1 point] Format String	25	☒	☐

[2 points] 2. Can you find a way to take control of the program by executing shellcode? Assume the shellcode is **32 bytes long**. List all assumptions and considerations that would make exploitation possible; **any missing assumption will cost points**. If exploitation is not possible, explain why. In your answer, be **exhaustive, concise, and straight to the point**.

Yes, exploitation is possible via the format string.

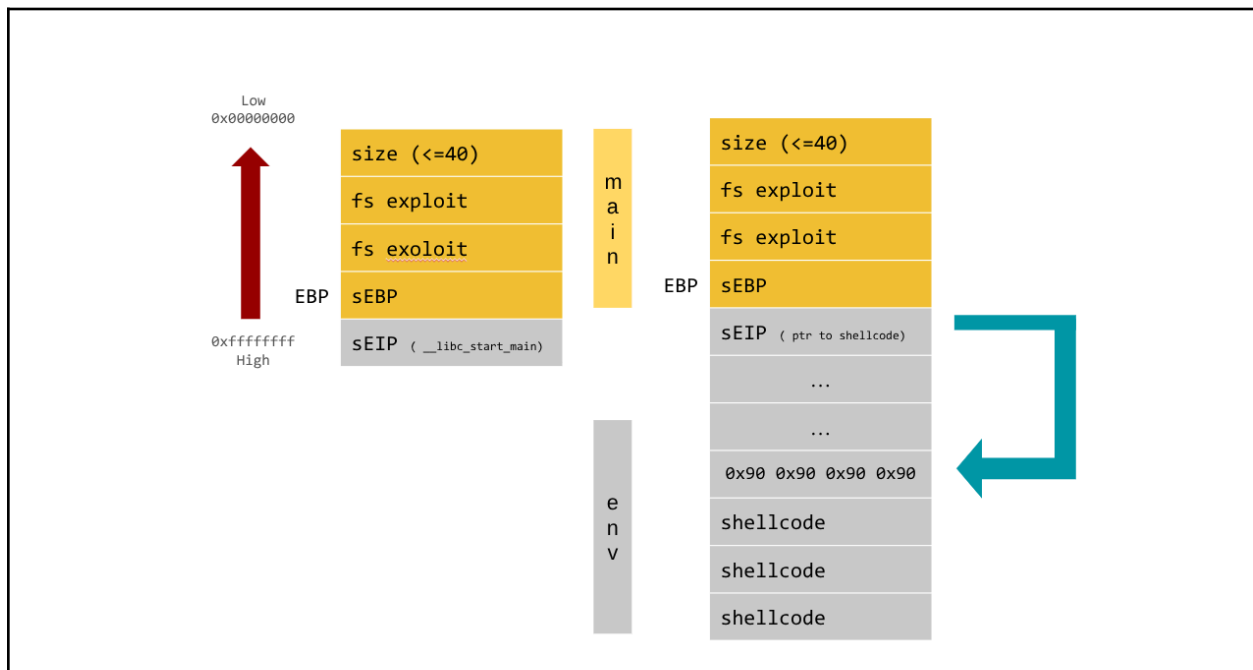
*If the provided size is not larger than the **alert** buffer (40 bytes), execution reaches **printf**, and the format string vulnerability can be exploited.*

The shellcode can be placed in the environment, since its large size makes it difficult to fit both the shellcode and the format-string exploit into the same buffer.

The buffer overflow cannot be exploited because, even if the boundary check is performed after the overflow, the program exits without returning, making it impossible to take control of the execution flow with the techniques seen in class.

[2 points] 3. Show the effect of the identified vulnerabilities on memory. If you have found an exploit that enables execution of the shellcode described in point 2, illustrate the **stack layout both before and after** the execution of the most relevant line(s) for that exploit, including:

- The **direction of memory growth** and the **relative address order**;
- The **name of each relevant allocated variable** in the stack frame;
- The **contents of relevant registers**, such as **EBP** and **ESP**;
- The **stack frames of the current function** and any **callers**;
- The **contents of the caller's frame**, including saved return addresses and local variables.



Exercise 4 - Web Vulnerabilities (6 points)

Nilo.com is a popular e-commerce platform where users can browse a catalog and purchase items. The relevant code of the application is reported below.

```
00 @app.route('/find_item', methods=['GET'])
01 def find_item(request, session):
02     if not is_session_valid(session):
03         return redirect(url_for("login"))
04     if "item_search" not in request.args:
05         item_search = ""
06     else:
07         item_search = request.args["item_search"]
08     if len(item_search) > 0 and not item_search.isalnum():
09         return render_template("home.html", error="Invalid item name: " + item_search)
10     search_q = "SELECT * FROM items WHERE name LIKE '%" + item_search + "%';"
11     results = execute_query_with_statements(search_q, item_search)
12     return render_template("find_item.html", items=results)
13
14 @app.route('/update_password', methods=['POST'])
15 def update_password(request, session):
16     if not is_session_valid(session):
17         return redirect(url_for("login"))
18     user_id = session["user_id"]
19     if "new_password" not in request.form:
20         return render_template("update_password.html", error="Please provide a new password!")
21     new_password = request.form["new_password"]
22     if "old_password" not in request.form:
23         return render_template("update_password.html", error="Please provide the old password!")
24     old_password = request.form["old_password"]
25     check_q = "SELECT * FROM users WHERE user_id=? AND password='" + old_password + "';"
26     results = execute_query_with_statements(check_q, user_id)
27     if len(results) != 1:
28         return render_template("update_password.html", error="Old password mismatch!")
29     update_q = "UPDATE users SET password = '?' WHERE user_id = '?'";
30     execute_query_with_statements(update_q, new_password, user_id)
31     return render_template("home.html", success="Password saved successfully!")
```

The relational database schema used by the website is the following:

Table: **users**

user_id (Primary Key) (int) (auto increment)	username (String)	password (String)	email (String)
--	----------------------	----------------------	-------------------

Table: **items**

item_id (Primary Key) (int) (auto increment)	name (string)	cost (float)
--	------------------	-----------------

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **is_session_valid** correctly checks if the user is logged in.
- The function **db_execute_query** executes queries without prepared statements.
- The function **db_execute_query_with_statements** executes queries with prepared statements.
- The function **render_template** does not perform any sanitization of the input.
- The **DBMS** does not support stacked queries (e.g., "SELECT ...; INSERT INTO ...;" fails if executed)
- The **DBMS** is case-sensitive; "Law" and "law" are two different users.
- The user's data is correctly sanitized when the user registers.

More details:

- The function **render_template** generates an HTML page from an .html file with optional parameters that are placed in the HTML code.
- The function **url_for** generates a URL for the provided page of the same domain. Optional query parameters of the URL can be specified as parameters.
- The function **redirect** redirects the user to the web page of the provided URL.
- The function **isalnum()** returns true if the string is alphanumeric, false otherwise.

4.1. [4 points] Answer the following questions:

For each SQL injection vulnerability you have identified, please provide the vulnerable line and the variable(s) that can be exploited for the injection. Moreover, explain why.

Present	Not present	Motivation
☒	☐	<i>Prepared statements are not correctly used</i>

If present:

Line	Variable(s)
<i>25</i>	<i>old_password</i>

For each XSS vulnerability you have identified, please provide the vulnerable variable, the XSS category, and where you would fix it. Moreover, explain why.

Present	Not present	Motivation
☒	☐	<i>item_search is not sanitized and it can contain an XSS payload</i>

If present:

Variable	XSS category			Lines to sanitize. It can be between two lines (e.g., 21-22) or on a single line. <u>Motivate the answer</u>
	Stored	Reflected	DOM-based	
<i>item_search</i>	☐	☒	☐	<i>08-09</i>
	☐	☐	☐	
	☐	☐	☐	

For each function, specify whether it is vulnerable to CSRF attacks. Moreover, explain why they are vulnerable.

Function	Vulnerable	Not vulnerable
<i>find_item</i>	☐	☒
<i>update_password</i>	☒	☐

Motivation:

find_item is not a state-changing action. Thus, no CSRF token is needed. update_password is a state-changing action without a CSRF token.

4.2. [2 points] Last night, somebody hacked the account of the user Law and managed to obtain his password. Can you reproduce the exploit? Explain all the steps you would take to reproduce the exploit, including the final code you would write.

The exploit is a blind SQL injection since the result of the query is not printed anywhere. The only option is to obtain a boolean information given by the login. If the login is successful, we inferred something. In this particular case, the following exploit:

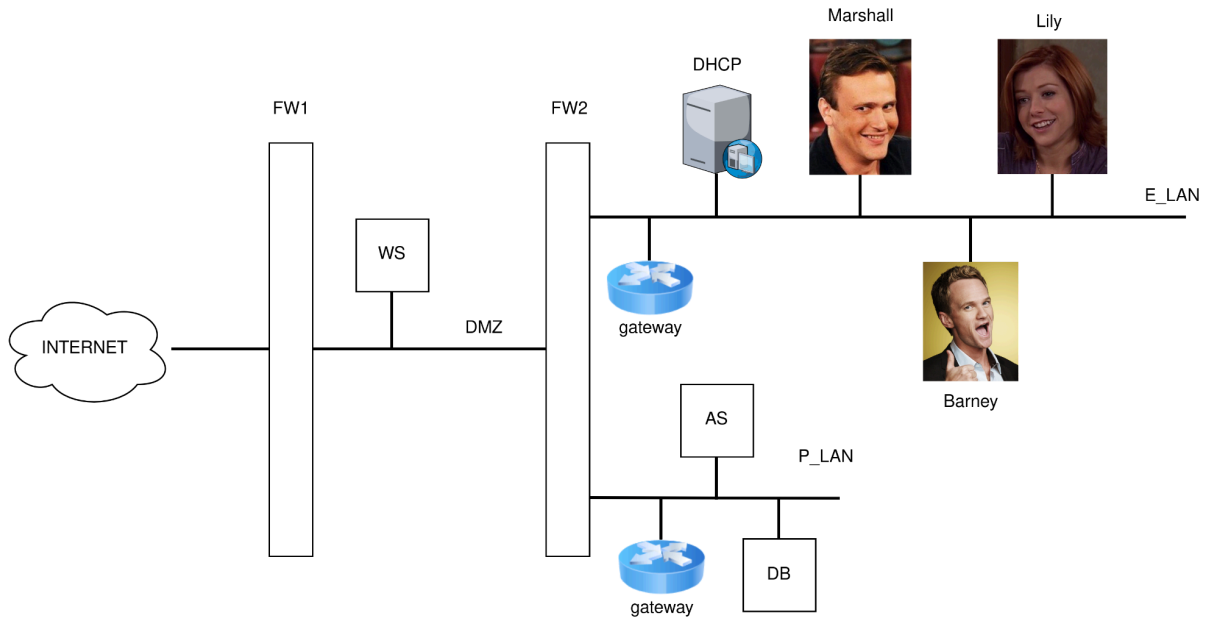
old_password = "invalid_password' OR username='Law' AND password LIKE 'a%'"

will result in a login only if the password of the user Law start with an a. If the login is not successful, we can try b% and so on. As soon as the login succeed, we leaked one character of the password and we move to the next one.

As a side effect, this exploit changes the attacker's current password.

Exercise 5 - Network Security (6 points)

To celebrate hiring their new attorney, Marshall Eriksen, Goliath National Bank threw a massive welcoming party organized by Barney Stinson. However, the morning after the event, the IT department issued an urgent alert reporting a major network breach. This is the network situation reconstructed by the IT department.



IT quickly flagged Lily as an intruder. Although she attended as Marshall's wife, her true goal was to hijack a workstation and launch a cyberattack to finally uncover the mystery of Barney's real job.

Assume that Barney needs to reach the Application Server (AS in the picture) over HTTP. IP's are not yet assigned; assignment is dynamic. Communication with Web Server and Application server adopts standard TCP protocol. Firewalls are correctly configured.

1. [2 points] What attack can she carry out among the ones reported in the table? Select the correct attack, if any.

- ☐ DNS Cache Poisoning
- ☐ TCP session hijacking
- ☒ **DHCP Poisoning**
- ☐ No attack can be done

Describe the assumptions and the steps Lily should perform to put in place the selected attack.
N.B. A general description of the attack, purely from theory, will not be considered a valid answer.

ASSUMPTIONS:

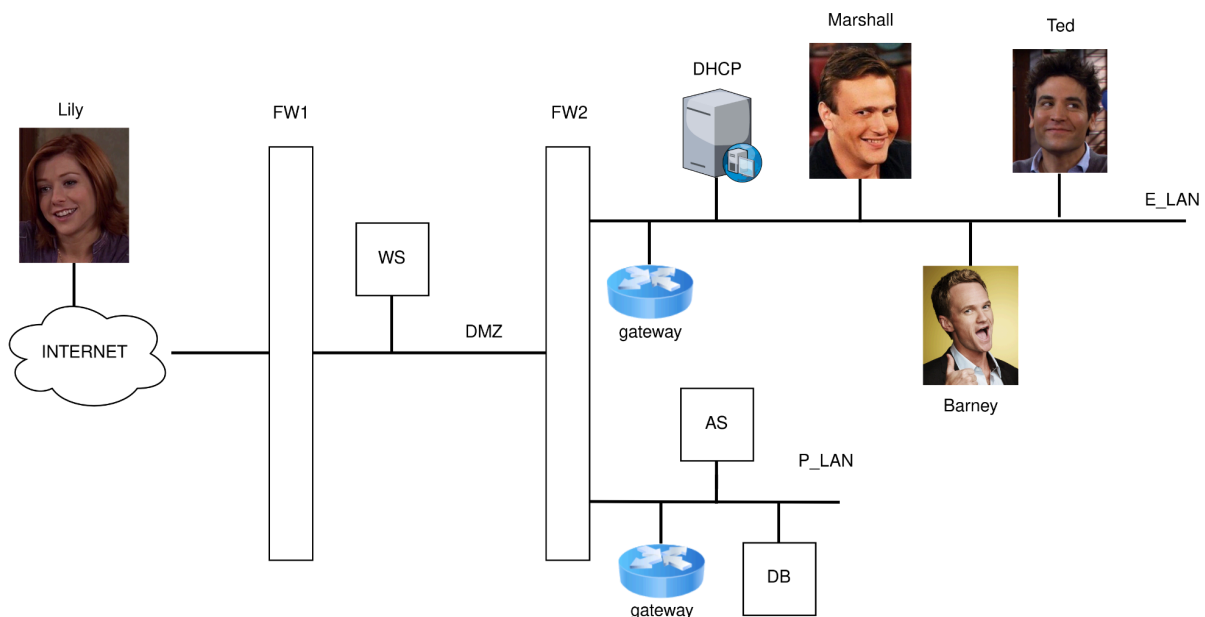
1. Lily and Barney are in the same LAN (Broadcast domain)
2. DHCP discover message is sent broadcast, thus it reaches Lily as well
3. DHCP is UDP based, without any authentication
4. Communication are HTTP (Barney's payload is in clear text)

ATTACK DESCRIPTION:

1. Barney needs to obtain it's IP, thus it first sends a DHCP discover in broadcast
2. Lily races the real DHCP server to respond to Barney, setting herself as the default gateway for Barney
3. Forwarding half-duplex MITM: all Barney messages are forwarded to the real gateway and from there to the AS, but Barney's messages are sniffed.

N.B. AS replies are NOT seen by Lily, as they are directly sent to Barney by the gateway.

2. [2 points] Promptly the IT guys kicked her out of the network by sanitizing the infected workstation. The picture below represents the new network situation, restored back to normality.



Can Lily carry out the same attack as before? Why? Why not?

Let's compare the assumptions of the previous answer: for the DHCP poisoning attack to work the attacker **MUST BE** in the same LAN as the victim. Since now Lily is outside the Employee LAN, she **CANNOT** carry out a DHCP poisoning attack.

3. [2 points] Months later, Goliath National Bank greenlit its new New York headquarters, tapping Ted Mosby, Barney's best friend, as head architect. To celebrate the design approval, Barney organized a company-wide "workcation" in Hawaii, namely a week of remote working followed by a massive two-day party.

To facilitate this, IT deployed a **Split Tunneling VPN**, allowing thousands of employees to access the internal Application Server while browsing the local web. However, upon their return, a workstation was found infected once again.

Explain the reasons why the infection happened.

Would a Full Tunneling VPN prevent the infection? Why? Why not? Would it have presented any downsides?.

By using **split tunneling**, GNB allowed employees to route specific work traffic through the secure VPN while letting general internet traffic pass through their unsecured hotel or public Wi-Fi. This allowed employees' computers to get infected, bypassing the corporate firewalls on their return to the office.

Full Tunnelling would prevent the infection as all the internet traffic would pass through the secure VPN, yet there would be the problem of traffic multiplication, causing ingestion in the network.

Exercise 6 - Malware (6 points)

```
int main(){
    CONTEXT ctx = {};
    char infoBuf[250];
    char username[] = "virtualbox";

    GetThreadContext(&ctx);

    if (ctx.Dr0 != 0 || ctx.Dr1 != 0 || ctx.Dr2 != 0 || ctx.Dr3 != 0)
        exit(0);

    GetUserNameA(infoBuf);

    if (strncmp(infoBuf, username, strlen(username)))
        exit(0);

    encrypt_all_files();
}
```

Some information useful in analyzing the code:

- **GetUserNameA(x)** writes the username that launched the process into the buffer **x**.
- **strcmp(x, y)** compares strings **x** and **y** (returns 0 if they're equal).
- **GetThreadContext** retrieves the context of the current thread, including register contents.
- **Dr0, Dr1, Dr2, Dr3** registers are used to handle hardware breakpoints. They are set to zero if the corresponding hardware breakpoint is not set, and to a non-zero value otherwise.
- **VirtualBox** is well-known software for running virtual machines.

[3 point] 1. What stealth techniques are implemented by the malware? Why were these techniques used? Please answer based on the provided code. Vague and textbook answers will not be considered valid.

Anti-virtualization (looking if analyst called their username virtualbox)

Anti-debugging (checking the use of hardware breakpoints)

Consider the following obfuscation wrapper.

```
import random

def main():
    random.seed(0)
    key = random.randbytes(16)
    out = encrypt("sample.exe", key)
    with open("stealth_sample.exe", "wb") as f:
        f.write(out)
```

Considering that

- **random** library generates random numbers using a PRNG initialized with a specific seed (so, with the same seed, you get the same random values)
- **random.seed(x)** sets the PRNG seed to **x**
- **random.randbytes(x)** gets **x** random bytes
- **encrypt(x, y)** function encrypts file **x** with key **y**

[2 point] 2. Which stealth technique is the malware author applying to the previous sample? Which analysis techniques is the technique trying to defeat?

Packing to defeat static analysis

[1 point] 3. Why is the implementation broken?

random.seed(0) will make the key always being the same for each sample.